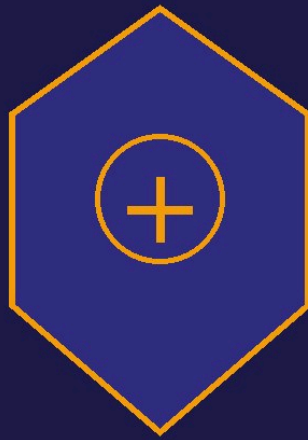




SECURITE WEB - EXPERT

Securite web

Architecture defensive, zero trust, supply chain, CI/CD et reponse a incident.

DanielCraft - Defense only - Pack Securite

Sommaire

1. Après l'intermédiaire : vision expert
 - Ce que tu vas apprendre
 - A retenir
2. Threat modeling (modélisation des menaces)
 - Méthode STRIDE (rappel défense)
 - Diagramme de flux
 - A retenir
3. Zero trust : ne jamais faire confiance par défaut
 - Principes
 - En pratique web
 - A retenir
4. OAuth 2 et OpenID Connect (vue défense)
 - Flux authorization code (recommande)
 - Pièges à éviter
 - A retenir
5. API : sécurité avancée
 - Checklist expert
 - Rate limiting avancé
 - A retenir
6. SSRF : Server-Side Request Forgery
 - Vecteurs
 - Défense
 - A retenir
7. Upload de fichiers : sécuriser
 - Règles
 - A retenir
8. Désérialisation et injections de gadgets
 - Défense
 - Web/API
 - A retenir
9. Supply chain : dépendances et CI
 - Actions
 - CI sécurise
 - A retenir
10. WAF, reverse proxy et edge
 - Couches
 - Ce que le WAF peut bloquer
 - Limites
 - A retenir
11. Réponse à incident

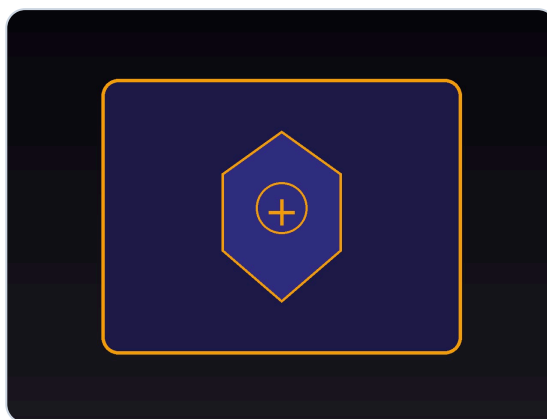
- [Phases](#)
 - [Communication](#)
 - [A retenir](#)
12. [Securite dans CI/CD](#)
- [Pipeline durcie](#)
 - [Deploiement](#)
 - [A retenir](#)
13. [Conteneurs et Kubernetes \(bases securite\)](#)
- [Docker](#)
 - [Kubernetes](#)
 - [A retenir](#)
14. [Rotation des secrets et gestion des cles](#)
- [Strategie](#)
 - [En cas de fuite](#)
 - [A retenir](#)
15. [Mini-projet : architecture securisee](#)
- [Exigences](#)
 - [Livrables](#)
 - [A retenir](#)
16. [Ce qu'il faut retenir](#)
- [Carte expert](#)
 - [Mindset](#)
 - [A retenir](#)
17. [Atelier : threat model express](#)
- [Scenario](#)
 - [Etapes](#)
 - [Correction type](#)
 - [A retenir](#)
18. [Atelier : audit checklist API](#)
- [Mission](#)
 - [A faire](#)
 - [Correction type](#)
 - [A retenir](#)
19. [Erreurs classiques niveau expert](#)
- [A retenir](#)
20. [Quiz securite expert](#)
- [Bareme](#)
 - [A retenir](#)
21. [Bravo !](#)
- [Tu sais maintenant](#)
 - [Et maintenant ?](#)

- [A retenir](#)

Après l'intermédiaire : vision expert



Du code durci à l'architecture.



Couverture Sécurité expert.

Tu maîtrises déjà XSS, CSRF, auth, headers, CSP. Le niveau **expert**, c'est penser **architecture**, **processus** et **resilience** : menacer modeler, répondre aux incidents, sécuriser la chaîne logicielle.

> **Astuce DanielCraft** - Expert = défense en profondeur + gouvernance. Toujours sans hacking offensif.

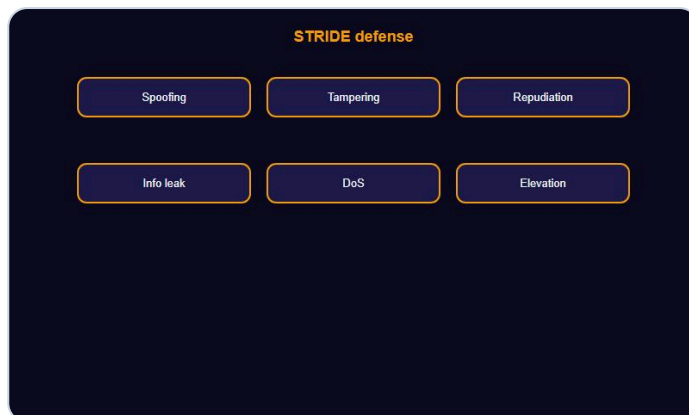
Ce que tu vas apprendre

- Threat modeling et zero trust.
- OAuth/OIDC, API avancée, SSRF, uploads.
- Supply chain, CI/CD, conteneurs.
- Réponse à incident et rotation des secrets.

A retenir

- Expert = sécurité dans le temps, pas seulement dans le code.
- Défense only, toujours.

Threat modeling (modelisation des menaces)



STRIDE en defense.

Avant de coder, tu identifies **qui** attaque, **quoi** proteger, **comment** entrer.

Methode STRIDE (rappel defense)

Lettre	Menace	Contre-mesure
S	Spoofing	Auth forte
T	Tampering	Integrite, signatures
R	Repudiation	Logs signes
I	Information disclosure	Chiffrement, ACL
D	Denial of service	Rate limit, CDN
E	Elevation of privilege	Moindre privilege

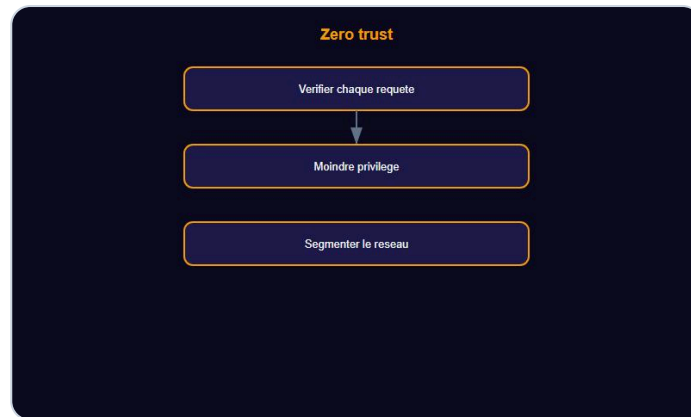
Diagramme de flux

1. Dessiner les acteurs (utilisateur, API, BDD, tiers).
 2. Lister les donnees sensibles en transit.
 3. Pour chaque fleche : quelle attaque ? quelle defense ?
- > **Astuce DanielCraft** - 30 minutes de threat model avant un sprint evitent des semaines de correctifs.

A retenir

- Threat model = carte des risques avant le code.
- STRIDE aide a ne rien oublier.

Zero trust : ne jamais faire confiance par défaut



Vérifier chaque requête.

Zero trust : chaque requête est vérifiée, même à l'intérieur du réseau.

Principes

1. **Vérifier explicitement** : auth + authz à chaque appel.
2. **Moindre privilège** : accès minimal, durée limitée.
3. **Supposer la compromission** : segmenter, chiffrer, monitorer.

En pratique web

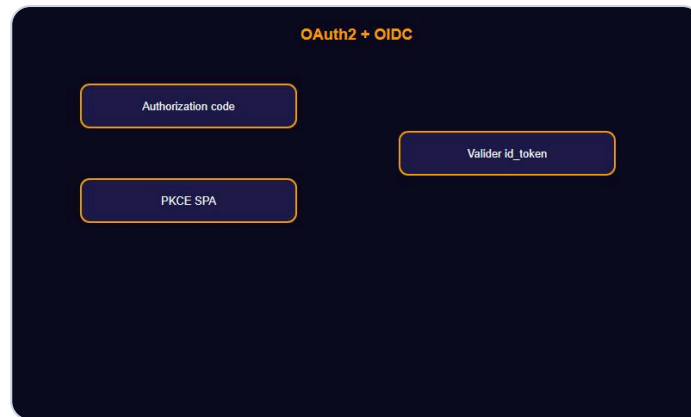
- mTLS entre microservices.
- JWT scopes fins par endpoint.
- Pas de confiance implicite au VPN seul.

```
Utilisateur -> API Gateway (auth) -> Service A (authz) -> BDD (role restreint)
```

A retenir

- Zero trust = vérifier partout, tout le temps.
- Le réseau interne n'est pas un bouclier magique.

OAuth 2 et OpenID Connect (vue defense)



OAuth2 + OIDC.

OAuth 2 delegate l'autorisation ; **OpenID Connect (OIDC)** ajoute l'identite (id_token).

Flux authorization code (recommande)

1. Redirection vers le fournisseur d'identite (IdP).
2. Code echange contre tokens **cote serveur** (jamais en front seul).
3. Validation : **iss** , **aud** , expiration, signature JWKS.

Pieges a eviter

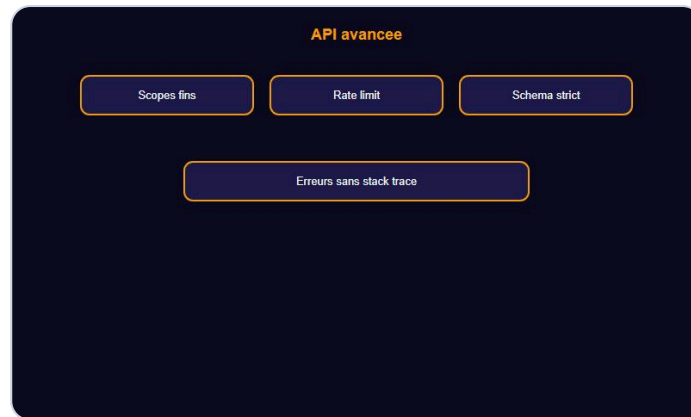
- Implicit flow en SPA sans PKCE.
- Stocker access_token en localStorage.
- Ne pas valider le **state** anti-CSRF.

> **Piege** - Un redirect_uri mal configure ouvre la porte au vol de code.

A retenir

- Authorization code + PKCE pour les apps modernes.
- Valider tous les claims du id_token.

API : securite avancee



API production-ready.

Une API en production doit resister aux abus, pas seulement aux bugs.

Checklist expert

- **Auth** : OAuth2, API keys rotantes, scopes.
- **Versioning** : deprecier sans casser la securite.
- **Pagination** : limiter la charge (max 100 items).
- **Idempotency-Key** sur POST sensibles.
- **Schema strict** : rejeter champs inconnus (mass assignment).

```
{  
  "error": "invalid_scope",  
  "message": "Scope orders:write requis"  
}
```

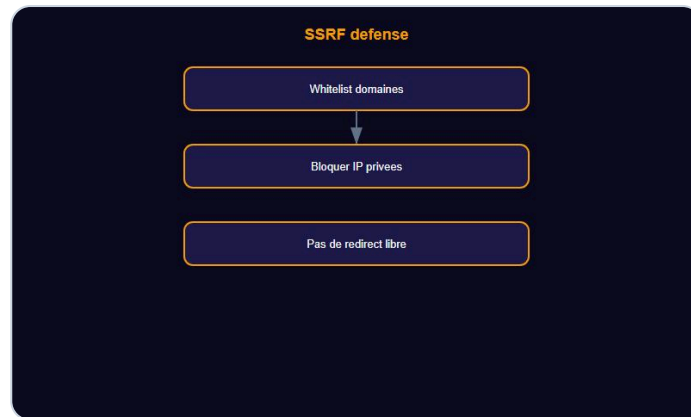
Rate limiting avance

- Par cle API + IP + endpoint.
- Slowdown progressif avant blocage.

A retenir

- API = surface d'attaque large : durcir chaque couche.
- Erreurs explicites pour le client, pas de stack trace.

SSRF : Server-Side Request Forgery



Bloquer les appels internes.

SSRF : l'attaquant force ton serveur à appeler une URL interne (metadata cloud, Redis local...).

Vecteurs

- Webhook URL fournie par l'utilisateur.
- Import d'image depuis une URL.
- Preview PDF / fetch proxy.

Defense

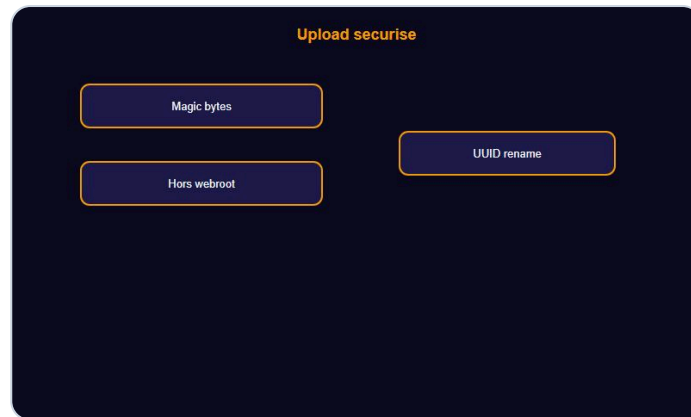
1. **Liste blanche** de domaines autorisés.
2. Bloquer IP privées (`10.x`, `169.254.x`, `127.0.0.1`).
3. Pas de redirections automatiques non contrôlées.
4. Réseau sortant en DMZ sans accès metadata.

> **Astuce DanielCraft** - Sur AWS, bloquer `169.254.169.254` en sortie sauf besoin explicite.

A retenir

- SSRF = ton serveur devient le proxy de l'attaquant.
- Whitelist + blocage réseau interne.

Upload de fichiers : securiser



Upload sans execution.

Un upload mal gere = execution de code ou fuite de donnees.

Regles

1. **Renommer** les fichiers (UUID), jamais le nom utilisateur.
2. **Verifier le type** : magic bytes, pas seulement l'extension.
3. **Stocker hors webroot** ou servir via CDN sans execution.
4. **Limiter taille** et nombre par utilisateur.
5. **Antivirus** optionnel sur fichiers sensibles.

```
ALLOWED = {"image/jpeg", "image/png", "application/pdf"}
if mime not in ALLOWED:
    raise ValidationError("Type non autorise")
```

A retenir

- Extension `.jpg` ne garantit rien : verifier le contenu.
- Jamais d'upload executable dans un dossier servi par le web server.

Deserialisation et injections de gadgets



JSON plutôt que binaire.

Deserialiser des données **non fiables** (pickle Python, Java ObjectInputStream, PHP unserialize) peut exécuter du code.

Defense

- **Eviter** la deserialisation de données utilisateur.
- Préférer **JSON** avec schema strict.
- Si obligatoire : bibliothèque durcie, liste blanche de classes.

Web/API

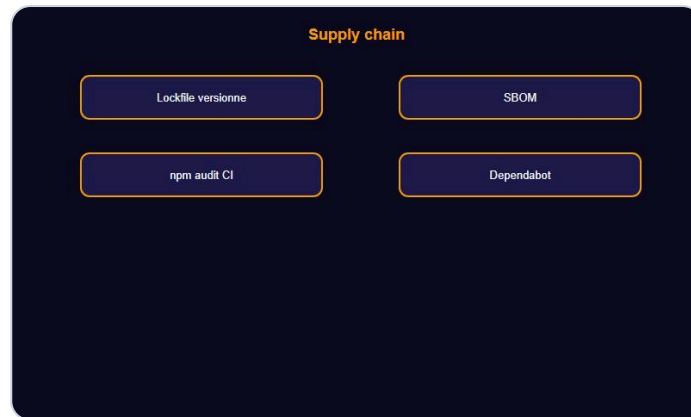
- Ne pas accepter de blobs binaires opaques sans validation.
- Signer les payloads internes (HMAC) si tu dois transporter des objets.

> **Piege** - `eval()` et `Function()` en JS sont des deserialisations déguisées.

A retenir

- Deserialisation = execution implicite : treat as code.
- JSON + validation > formats binaires legacy.

Supply chain : dependances et CI



Audit des dependances.

Ton code depend de milliers de paquets. Une dependance compromise = ta prod compromise.

Actions

- **Lockfile** (`package-lock.json` , `poetry.lock`) versionne.
- `npm audit` / `pip-audit` / **Dependabot** en CI.
- **SBOM** (Software Bill of Materials) pour la tracabilite.
- Verifier signatures et checksums des releases.

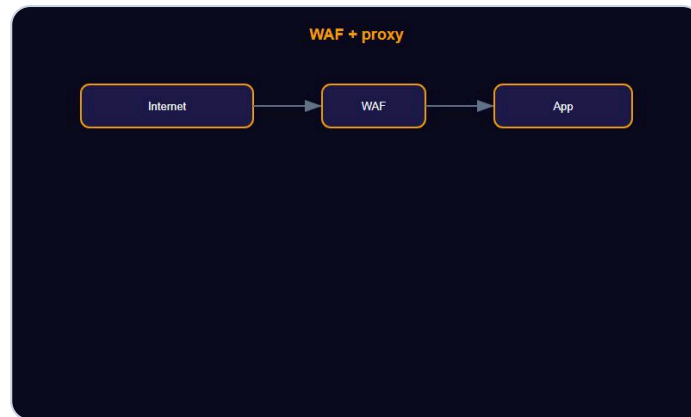
CI securise

- Secrets dans le vault CI, pas dans les logs.
- Branches protegees, revue obligatoire.
- Build reproductible, images signees.

A retenir

- Supply chain = partie de ta surface d'attaque.
- Audit automatique a chaque merge.

WAF, reverse proxy et edge



Filtrage edge.

Le **WAF** (Web Application Firewall) filtre le trafic HTTP avant qu'il n'atteigne ton app.

Couches

```
Internet -> CDN/WAF -> Load balancer -> App -> BDD
```

Ce que le WAF peut bloquer

- Signatures SQLi/XSS connues.
- Bots et scans automatisés.
- Geo-blocking si pertinent.

Limites

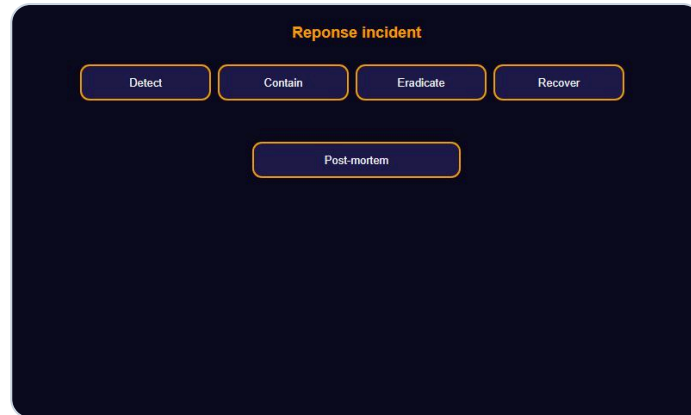
- Faux positifs sur API JSON.
- Règles custom pour ton métier.
- Le WAF **complete** le code, ne le remplace pas.

> **Astuce DanielCraft** - Mode detection d'abord, blocage après tuning.

A retenir

- WAF = filtre edge, pas substitute au code sécurisé.
- TLS termine au proxy avec config moderne.

Reponse a incident



Runbook et post-mortem.

Quand (pas si) un incident arrive, la preparation fait la difference.

Phases

1. **Preparation** : runbook, contacts, backups testes.
2. **Detection** : alertes, SIEM, logs centralises.
3. **Containment** : isoler, revoquer tokens, couper acces.
4. **Eradication** : patch, rotation secrets.
5. **Recovery** : restore, monitoring renforce.
6. **Post-mortem** : blameless, actions correctives.

Communication

- Informer le DPO si donnees personnelles.
- Message utilisateur clair sans details techniques exploitables.

A retenir

- Runbook ecrit avant l'incident.
- Post-mortem = amelioration continue.

Securite dans CI/CD



SAST, scan image, secrets.

La pipeline est une porte d'entree : protege-la comme la prod.

Pipeline durcie

- **SAST** (analyse statique) sur chaque PR.
- **DAST** periodique sur staging.
- Scan des **images Docker** (Trivy, Grype).
- Pas de secrets dans le Dockerfile ou les variables en clair.

```
# Exemple etape CI
- run: npm audit --audit-level=high
- run: trivy image --exit-code 1 myapp:${{ github.sha }}
```

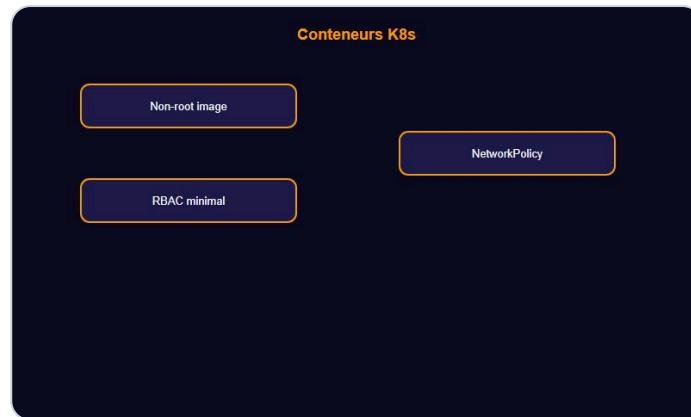
Deploiement

- Environnements isoles (dev/staging/prod).
- Approbation manuelle pour prod.
- Rollback automatise teste.

A retenir

- CI/CD compromis = prod compromise.
- Scans automatiques non negociables.

Conteneurs et Kubernetes (bases securite)



Non-root, RBAC, NetworkPolicy.

Docker et K8s apportent de nouvelles surfaces : images, orchestrateur, reseau.

Docker

- Image **non-root** (USER 1000).
- Read-only filesystem si possible.
- Pas de `--privileged` sans raison.

Kubernetes

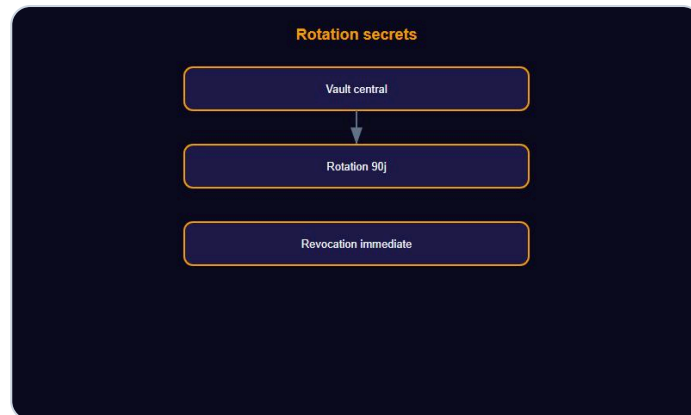
- **RBAC** minimal par namespace.
- **NetworkPolicy** : isoler les pods.
- Secrets K8s chiffrés at rest (etcd encryption).
- Pas de `cluster-admin` pour les apps.

> **Piege** - Monter le socket Docker dans un conteneur = game over.

A retenir

- Conteneur != sandbox magique : durcir image + orchestrateur.
- Moindre privilege partout.

Rotation des secrets et gestion des cles



Vault et rotation.

Un secret qui ne tourne jamais reste une bombe a retardement.

Strategie

- **Rotation automatique** (90 jours max pour cles API).
- **Vault** (HashiCorp, AWS Secrets Manager) avec audit trail.
- Separation : cle dev != staging != prod.

En cas de fuite

1. Revoquer immediatement.
2. Generer nouvelle cle.
3. Deployer sans downtime (double cle temporaire).
4. Analyser les logs d'usage de l'ancienne cle.

```
# Jamais en clair dans l'historique shell
export API_KEY=$(vault read -field=key secret/prod/api)
```

A retenir

- Rotation = routine, pas urgence seule.
- Vault centralise et journalise.

Mini-projet : architecture securisee



Architecture multi-tenant.

Objectif : concevoir (sur papier ou schema) une API SaaS multi-tenant avec checklist expert.

Exigences

- Auth OIDC + scopes par tenant.
- Rate limit par tenant et endpoint.
- Upload avatars (images only, S3 prive).
- Logs sans PII, alertes echecs auth.
- CI : audit deps + scan image.

Livrables

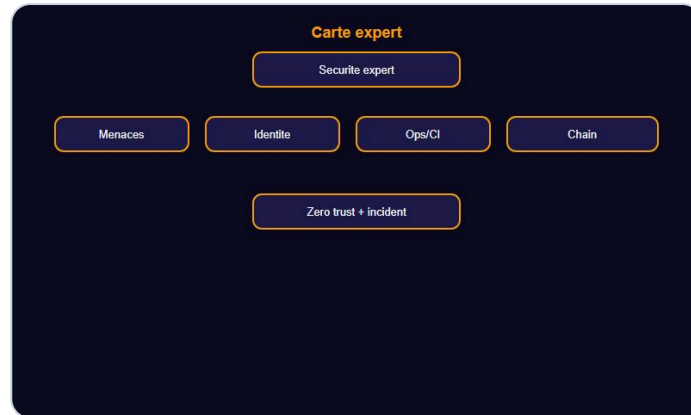
1. Diagramme threat model (STRIDE sur 3 flux).
2. Liste headers HTTP + CSP proposee.
3. Runbook incident (1 page).

> **Astuce DanielCraft** - Presente le schema a un collegue : s'il comprend les flux, c'est bon.

A retenir

- Mini-projet = synthese architecture + processus.
- Documenter avant d'implementer.

Ce qu'il faut retenir



Carte expert.

Carte expert

- **Menaces** : STRIDE, zero trust, SSRF, uploads.
- **Identité** : OAuth/OIDC, rotation secrets.
- **Ops** : WAF, CI/CD, conteneurs, incident response.
- **Chain** : supply chain, SBOM, audits.

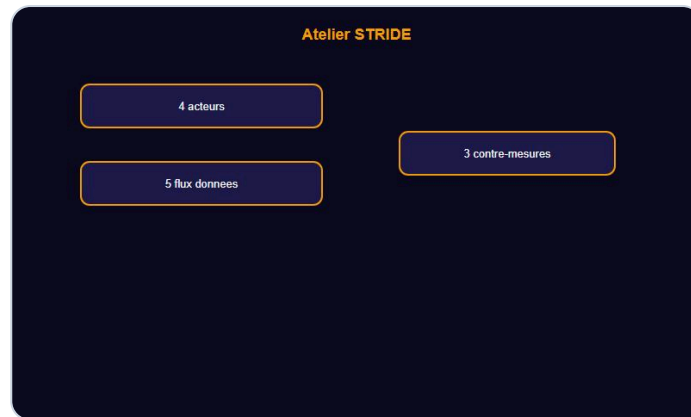
Mindset

- Defense in depth + gouvernance.
- Mesurer, alerter, améliorer (post-mortem).

A retenir

- Expert = sécurité durable, pas un patch unique.

Atelier : threat model express



Atelier STRIDE.

Duree : 25 minutes.

Scenario

Application de prise de rendez-vous medicaux : patient, medecin, admin, API, BDD PostgreSQL, envoi SMS tiers.

Etapas

1. Dessiner les 4 acteurs et 5 flux de donnees.
2. Appliquer STRIDE sur le flux « patient reserve un creneau ».
3. Lister 3 contre-mesures prioritaires.

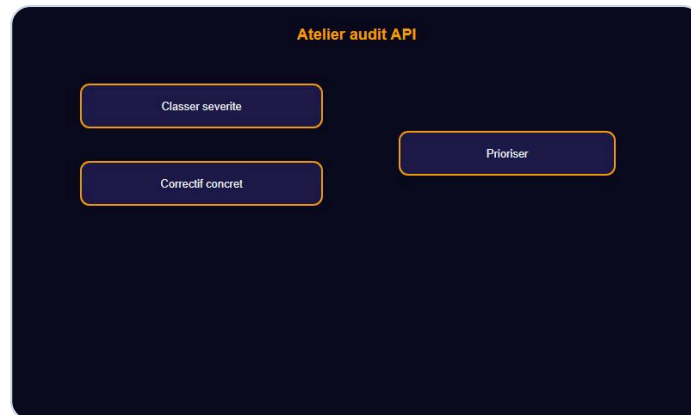
Correction type

- Spoofing patient -> auth MFA optionnelle.
- Tampering creneau -> validation serveur + verrou optimistic.
- Info disclosure -> chiffrement BDD + ACL par role.

A retenir

- 25 min suffisent pour un premier threat model utile.

Atelier : audit checklist API



Atelier audit API.

Duree : 20 minutes.

Mission

Auditer une API REST fictive `/api/v1/orders` avec ces constats :

- JWT en localStorage.
- Pas de rate limit sur POST.
- Erreurs 500 avec stack trace.
- CORS `*`.

A faire

1. Classer chaque point : critique / majeur / mineur.
2. Proposer le correctif concret.
3. Prioriser l'ordre de correction.

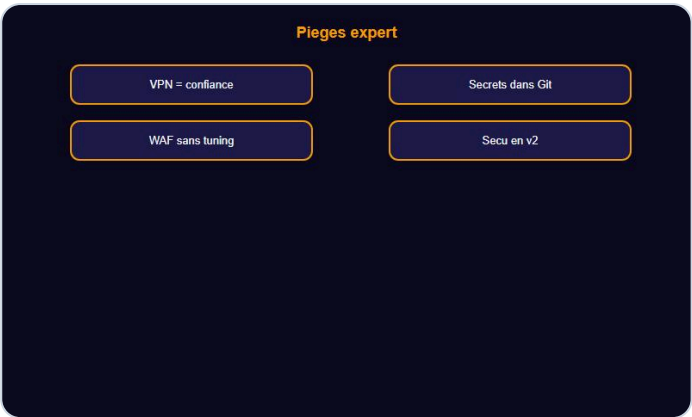
Correction type

1. JWT -> cookie HttpOnly + refresh rotation.
2. Rate limit -> 100 req/min par token.
3. Erreurs -> JSON generique + log serveur.
4. CORS -> origines explicites.

A retenir

- Audit = prioriser par impact x facilite.

Erreurs classiques niveau expert



Pieges organisationnels.

Meme les seniors tombent dans ces pieges.

Erreur	Pourquoi c'est grave	Fix
Confiance au VPN seul	Lateral movement interne	Zero trust
WAF sans tuning	Faux sentiment ou blocages	Detection puis blocage
Secrets dans K8s YAML Git	Fuite permanente	Vault + sealed secrets
Ignorer supply chain	Compromission transitive	Lockfile + audit CI
Pas de runbook incident	Panique, delai x10	Documenter maintenant

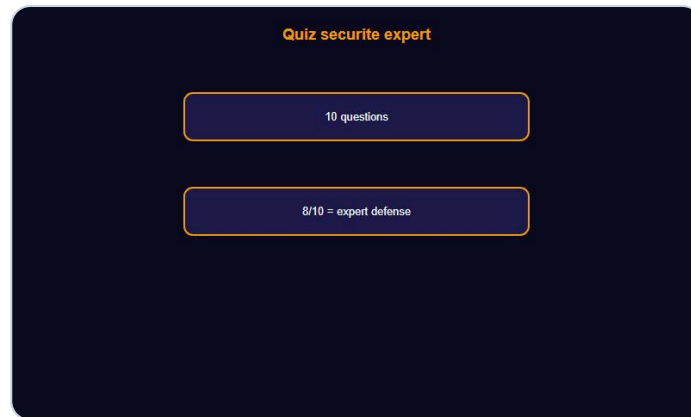
> **Piege** - « On fera la securite en v2 » : la v2 n'arrive jamais avant la breach.

A retenir

- Expert = eviter les raccourcis organisationnels.

QUIZ

Quiz securite expert



Quiz securite expert.

Reponds mentalement, puis verifie.

1. 1. Que signifie le **S** de STRIDE ?
2. 2. Quel flux OAuth est recommande pour une SPA ?
3. 3. Comment limiter le **SSRF** sur un webhook URL ?
4. 4. Pourquoi eviter pickle sur des donnees utilisateur ?
5. 5. Que faire en **premiere** heure d'un incident de fuite de tokens ?
6. 6. Role du **WAF** vs code applicatif ?
7. 7. Trois regles pour un upload de fichier securise ?
8. 8. Qu'est-ce qu'un **SBOM** ?
9. 9. Pourquoi rotation des secrets tous les 90 jours ?
10. 10. Un element cle d'un post-mortem blameless ?

Bareme

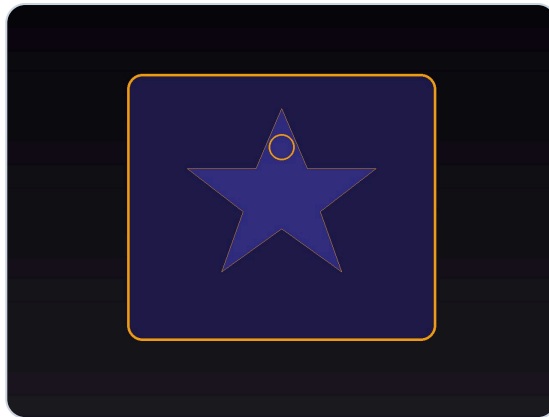
- 8/10 ou plus : solide niveau expert defense.
- 6-7 : relire menaces + ops.
- Moins : reprendre intermediaire puis revenir ici.

A retenir

- Le quiz valide la vision globale, pas la memorisation.

FINAL

Bravo !



Bravo. Tu es pret pour la prod.

Tu as parcouru **Securite web - Expert** : threat modeling, zero trust, OAuth, API avancee, supply chain, CI/CD, conteneurs, incidents.

Tu sais maintenant

- Concevoir une architecture defensive durable.
- Integrer securite dans la chaine devops.
- Reagir quand quelque chose tourne mal.

Et maintenant ?

- Applique un threat model sur ton projet actuel.
- Mets en place audit deps + rotation secrets.
- Partage le runbook incident avec ton equipe.

> **DanielCraft** - Expert ne veut pas dire parano : ca veut dire prepare. Tu l'es.

A retenir

- Securite = voyage, pas destination. Continue a mesurer et ameliorer.

Tu conçois. Tu proteges. Tu reagis.